

```
In [41]: import pandas as pd
```

```
In [3]: # configure pandas to display more columns
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
```

```
In [5]: df = pd.read_csv("Superstore.csv")
```

```
In [11]: df.head()    # Shows first 5 rows
```

Out[11]:

	Ship Mode	Segment	Country	City	State	Postal Code	Region	Ca
0	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Fu
1	Second Class	Consumer	United States	Henderson	Kentucky	42420	South	Fu
2	Second Class	Corporate	United States	Los Angeles	California	90036	West	Of Su
3	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Fu
4	Standard Class	Consumer	United States	Fort Lauderdale	Florida	33311	South	Of Su

```
In [12]: df.tail()
```

Out[12]:

	Ship Mode	Segment	Country	City	State	Postal Code	Region
9989	Second Class	Consumer	United States	Miami	Florida	33180	South
9990	Standard Class	Consumer	United States	Costa Mesa	California	92627	West
9991	Standard Class	Consumer	United States	Costa Mesa	California	92627	West
9992	Standard Class	Consumer	United States	Costa Mesa	California	92627	West
9993	Second Class	Consumer	United States	Westminster	California	92683	West

```
In [13]: df.info() # Shows last 5 rows
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9994 entries, 0 to 9993
Data columns (total 13 columns):
#   Column          Non-Null Count  Dtype  
---  -
0   Ship Mode       9994 non-null  object  
1   Segment         9994 non-null  object  
2   Country         9994 non-null  object  
3   City            9994 non-null  object  
4   State           9994 non-null  object  
5   Postal Code     9994 non-null  int64   
6   Region          9994 non-null  object  
7   Category        9994 non-null  object  
8   Sub-Category    9994 non-null  object  
9   Sales           9994 non-null  float64  
10  Quantity        9994 non-null  int64   
11  Discount        9994 non-null  float64  
12  Profit          9994 non-null  float64  
dtypes: float64(3), int64(2), object(8)
memory usage: 1015.1+ KB
```

```
In [18]: # Get statistics for numerical columns
print("\nStatistical Summary (df.describe()):")
print(df.describe())
```

```
Statistical Summary (df.describe()):
```

	Postal Code	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000	9994.000000
mean	55190.379428	229.858001	3.789574	0.156203	28.656896
std	32063.693350	623.245101	2.225110	0.206452	234.260108
min	1040.000000	0.444000	1.000000	0.000000	-6599.978000
25%	23223.000000	17.280000	2.000000	0.000000	1.728750
50%	56430.500000	54.490000	3.000000	0.200000	8.666500
75%	90008.000000	209.940000	5.000000	0.200000	29.364000
max	99301.000000	22638.480000	14.000000	0.800000	8399.976000

```
In [19]: print("Check for missing values")
missing = df.isnull().sum()
print(missing)
```

```
Check for missing values
Ship Mode      0
Segment       0
Country        0
City           0
State          0
Postal Code    0
Region         0
Category       0
Sub-Category   0
Sales          0
Quantity       0
Discount       0
Profit         0
dtype: int64
```

In [25]:

```
print("\n" + "="*80)
print("% of missing values")
missing_percent = (df.isnull().sum() / len(df)) * 100
print(missing_percent)
```

```
=====
===
% of missing values
Ship Mode      0.0
Segment        0.0
Country        0.0
City           0.0
State          0.0
Postal Code    0.0
Region         0.0
Category       0.0
Sub-Category   0.0
Sales          0.0
Quantity       0.0
Discount       0.0
Profit         0.0
dtype: float64
```

In [30]:

```
print("\n" + "="*80)
print("Check Duplicates Rows")
print("="*80)
duplicates = df.duplicated().sum()
print("Number of Duplicates rows:", duplicates)
```

```
=====
===
Check Duplicates Rows
=====
===
Number of Duplicates rows: 17
```

```
In [33]: df_cleaned = df.drop_duplicates()
print(df_cleaned)
```

	Ship Mode	Segment	Country	City	State
\					
0	Second Class	Consumer	United States	Henderson	Kentucky
1	Second Class	Consumer	United States	Henderson	Kentucky
2	Second Class	Corporate	United States	Los Angeles	California
3	Standard Class	Consumer	United States	Fort Lauderdale	Florida
4	Standard Class	Consumer	United States	Fort Lauderdale	Florida
...	...	...	...	...	...
9989	Second Class	Consumer	United States	Miami	Florida
9990	Standard Class	Consumer	United States	Costa Mesa	California
9991	Standard Class	Consumer	United States	Costa Mesa	California
9992	Standard Class	Consumer	United States	Costa Mesa	California
9993	Second Class	Consumer	United States	Westminster	California

	Postal Code	Region	Category	Sub-Category	Sales	Quantity	\
0	42420	South	Furniture	Bookcases	261.9600	2	
1	42420	South	Furniture	Chairs	731.9400	3	
2	90036	West	Office Supplies	Labels	14.6200	2	
3	33311	South	Furniture	Tables	957.5775	5	
4	33311	South	Office Supplies	Storage	22.3680	2	
...	...	...	...	...	...	...	...
9989	33180	South	Furniture	Furnishings	25.2480	3	
9990	92627	West	Furniture	Furnishings	91.9600	2	
9991	92627	West	Technology	Phones	258.5760	2	
9992	92627	West	Office Supplies	Paper	29.6000	4	
9993	92683	West	Office Supplies	Appliances	243.1600	2	

	Discount	Profit
0	0.00	41.9136
1	0.00	219.5820
2	0.00	6.8714
3	0.45	-383.0310
4	0.20	2.5164
...	...	...
9989	0.20	4.1028
9990	0.00	15.6332
9991	0.20	19.3932
9992	0.00	13.3200
9993	0.00	72.9480

[9977 rows x 13 columns]

```
In [34]: print(df.dtypes)
```

```
Ship Mode      object
Segment        object
Country         object
City            object
State           object
Postal Code     int64
Region          object
Category        object
Sub-Category    object
Sales           float64
Quantity        int64
Discount        float64
Profit          float64
dtype: object
```

```
In [38]: print("Convert 'Postal Code' from Integer to String/Object")
df['Postal Code'] = df['Postal Code'].astype(str)
print(df.dtypes)
```

```
Convert 'Postal Code' from Integer to String/Object
Ship Mode      object
Segment        object
Country         object
City            object
State           object
Postal Code     object
Region          object
Category        object
Sub-Category    object
Sales           float64
Quantity        int64
Discount        float64
Profit          float64
dtype: object
```

```
In [40]: print("--- Statistical Summary ---")
df.describe()
```

```
--- Statistical Summary ---
```

Out[40]:

	Sales	Quantity	Discount	Profit
count	9994.000000	9994.000000	9994.000000	9994.000000
mean	229.858001	3.789574	0.156203	28.656896
std	623.245101	2.225110	0.206452	234.260108
min	0.444000	1.000000	0.000000	-6599.978000
25%	17.280000	2.000000	0.000000	1.728750
50%	54.490000	3.000000	0.200000	8.666500
75%	209.940000	5.000000	0.200000	29.364000
max	22638.480000	14.000000	0.800000	8399.976000

```
In [42]: import matplotlib.pyplot as plt
import seaborn as sns
sns.set_theme(style="whitegrid")
print("Visualization libraries loaded successfully")
```

Visualization libraries loaded successfully

In [45]:

```
# Group data to see Total Sales & Profit
category_analysis = df.groupby('Category')[['Sales',
'Profit']].sum().sort_values(by='Sales', ascending=False)

print("--- Sales and Profit by Category ---")
print(category_analysis)

plt.figure(figsize=(12,6))
sub_category_sales = df.groupby('Sub-Category')
['Sales'].sum().sort_values(ascending=False).head(10)

# Horizontal Bar Chart
sns.barplot(x=sub_category_sales.values, y=sub_category_sales.index,
palette="viridis")

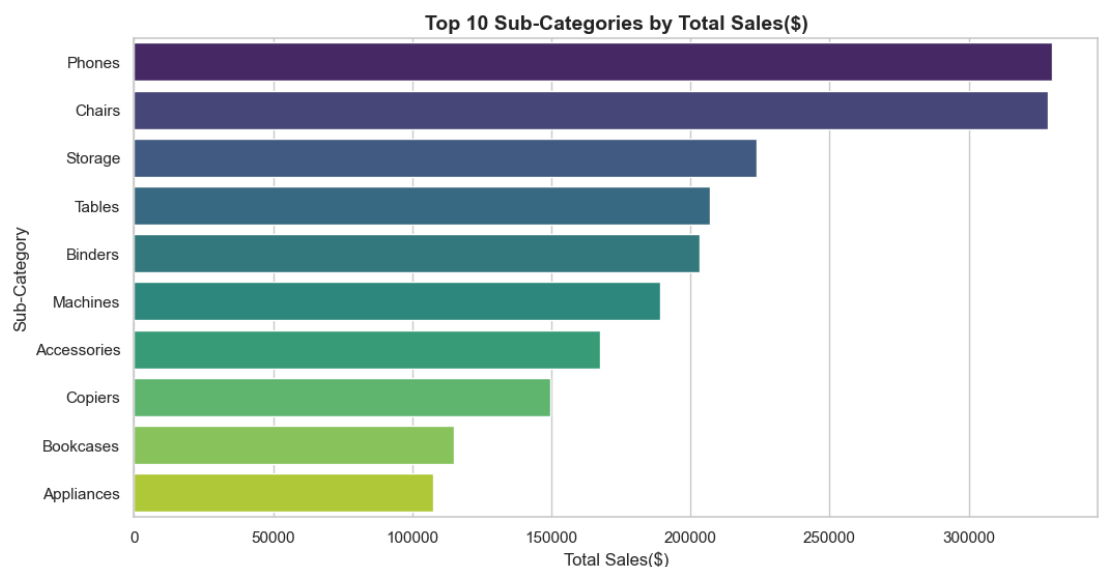
plt.title('Top 10 Sub-Categories by Total Sales($)', fontsize=14,
fontweight='bold')
plt.xlabel('Total Sales($)')
plt.ylabel('Sub-Category')
plt.show()
```

```
--- Sales and Profit by Category ---
              Sales      Profit
Category
Technology    836154.0330  145454.9481
Furniture     741999.7953   18451.2728
Office Supplies 719047.0320  122490.8008
```

C:\Users\vivek\AppData\Local\Temp\ipykernel_17400\3908146191.py:11:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `y` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(x=sub_category_sales.values, y=sub_category_sales.index,
palette="viridis")
```



In [47]:

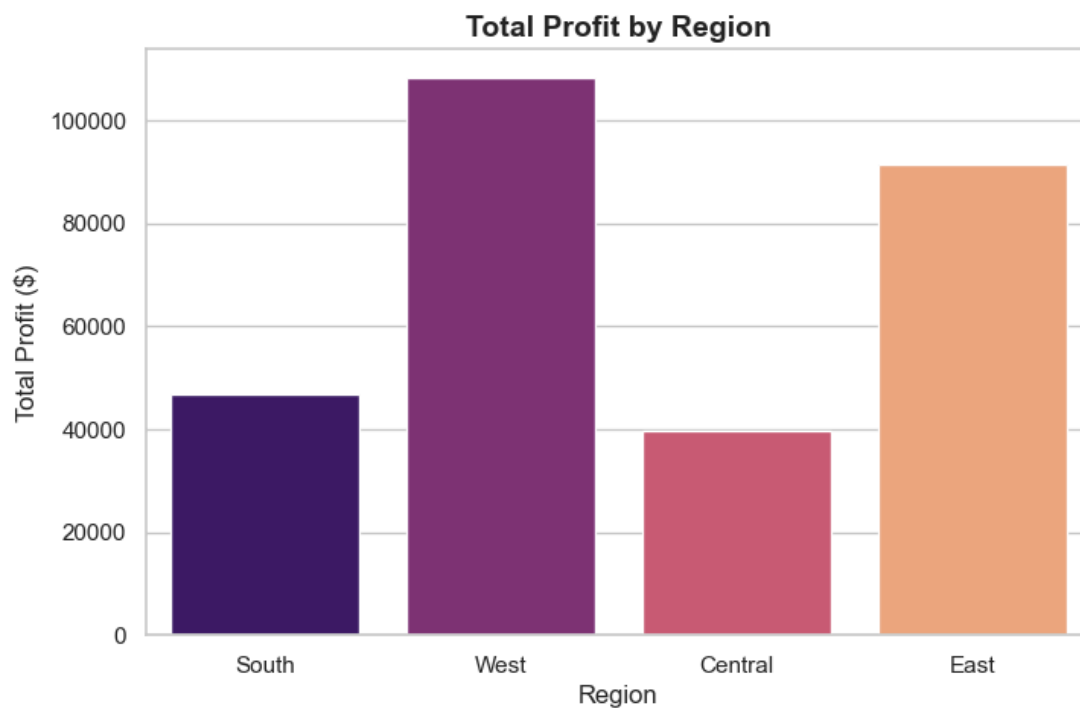
```
# Plot Total Profit generated by Region
plt.figure(figsize=(8, 5))
# estimator=sum to add all profit in each region
sns.barplot(data=df, x='Region', y='Profit', estimator=sum, errorbar=None,
palette="magma")

plt.title('Total Profit by Region', fontsize=14, fontweight='bold')
plt.xlabel('Region')
plt.ylabel('Total Profit ($)')
plt.show()
```

C:\Users\vivek\AppData\Local\Temp\ipykernel_17400\1606072597.py:4:
FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.

```
sns.barplot(data=df, x='Region', y='Profit', estimator=sum,
errorbar=None, palette="magma")
```



In [50]:

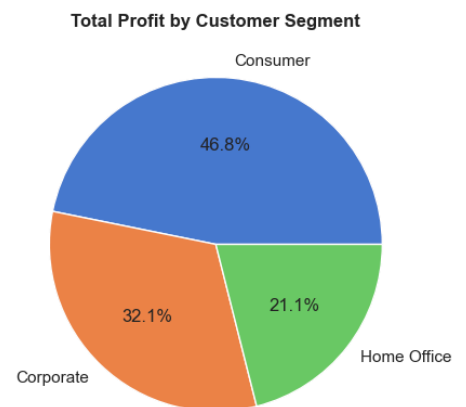
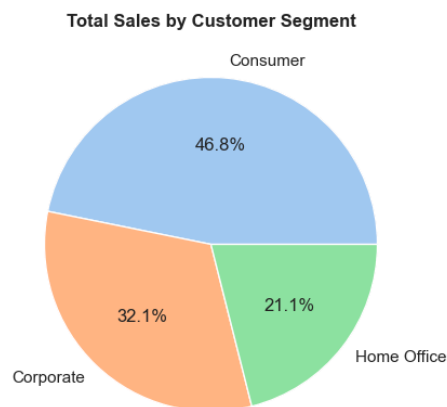
```
# Group by Segment to calculate Total Sales and Profit
segment_analysis = df.groupby('Segment')[['Sales', 'Profit']].sum()

# Plot Sales and Profit
fig, axes = plt.subplots(1, 2, figsize=(14, 5))

# Sales Pie Chart
axes[0].pie(segment_analysis['Profit'], labels=segment_analysis.index,
autopct='%1.1f%%', colors=sns.color_palette("pastel"))
axes[0].set_title('Total Sales by Customer Segment', fontweight='bold')

# Profit Pie Chart
axes[1].pie(segment_analysis['Profit'], labels=segment_analysis.index,
autopct='%1.1f%%', colors=sns.color_palette("muted"))
axes[1].set_title('Total Profit by Customer Segment', fontweight='bold')

plt.show()
```

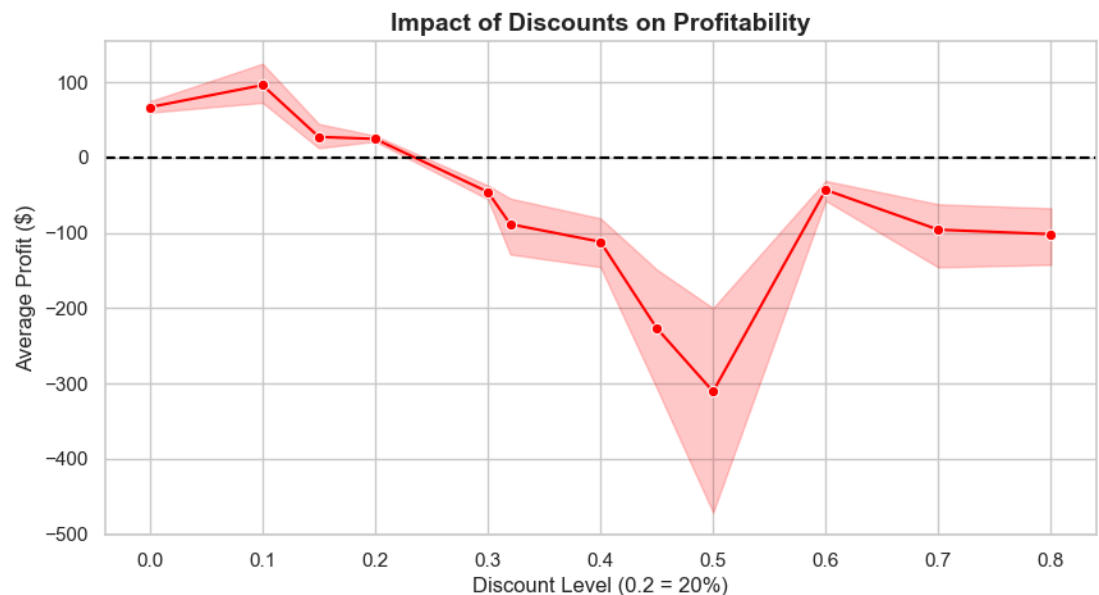


In [51]:

```
# Discount vs Profit
plt.figure(figsize=(10, 5))

# Creating Line plot showing relationship between Discount and Profit
sns.lineplot(data=df, x='Discount', y='Profit', color="red", marker="o")

plt.title('Impact of Discounts on Profitability', fontsize=14, fontweight='bold')
plt.xlabel('Discount Level (0.2 = 20%)')
plt.ylabel('Average Profit ($)')
plt.axhline(0, color='black', linestyle='--') # adds a line at $0 profit
plt.show()
```



Executive Summary & Business Recommendations

Based on the exploratory data analysis of the Superstore dataset, here are the key findings:

1. **Product Strategy:** 'Phones' and 'Chairs' drive the highest total revenue. However, the company should evaluate the profit margins on top-selling items to ensure high volume translates to high returns.

2. **Regional Focus:** The West region is the most profitable, whereas the Central region struggles to generate proportionate profit despite strong sales. Investigating operational costs in the Central region is recommended.

3. **Discount Policy Restructuring (Critical Insight):** The data shows a severe negative correlation between high discounts and profitability. Any discount above 20% (0.2) consistently results in a net loss. The business must immediately cap discounts at 20% to protect the bottom line.

4. **Target Audience:** The 'Consumer' segment is the largest driver of both sales and profit (over 50%), followed by 'Corporate'. Marketing budgets should be allocated accordingly.

In []:

Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).